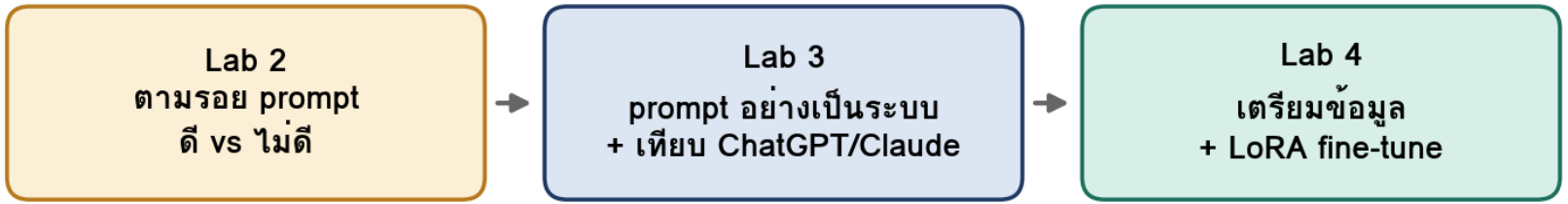
ช่วงบ่าย: ลงมือทำ ตามรอย prompt · prompt อย่างเป็นระบบ · fine-tune ด้วย LoRA

LLM & Agentic AI · สัปดาห์ 3 · ช่วงบ่าย (ปฏิบัติบน Google Colab)

ดร.ธีรพงศ์ อรชร

ภาพรวม

บ่ายนี้ทำอะไร และเชื่อมกับตอนเช้าตรงไหน



ทุกแล็บใช้ Typhoon2-1B บน Colab T4 · Lab 1 (ผ่าโมเดล) ทำไปแล้วตอนเช้า

- Lab 2 ↔ ส่วนที่ 3 ตอนเช้า (เส้นทาง prompt 6 ขั้น)
- Lab 3 ↔ หลักการ prompt 5 ข้อ
- Lab 4 ↔ ส่วนที่ 2 (LoRA, chat template, dev/test)

Lab 0

Lab 0 · เตรียม Google Colab (ทำครั้งเดียว ใช้ทุกแล็บ)

1 · เปิด GPU

Runtime → Change runtime type → T4 GPU

ถ้าไม่เปิด โมเดล 1B จะช้ามากและ Lab 4 เทรนไม่ได้

2 · ติดตั้ง

เซลล์แรกของทุก notebook ติดตั้งให้เอง (transformers, accelerate · Lab 4 เพิ่ม peft, datasets)

ไม่ถึง 1 นาที ไม่ต้อง restart

3 · โหลดโมเดล

เซลล์ถัดมาโหลด Typhoon2-1B แบบ fp16

ครั้งแรกดาวน์โหลด ≈ 2.5 GB ใช้ 1–3 นาที

Lab 2 โหลดแบบ attention "eager" เพื่อดึง attention ออกมาดูได้

4 · ถ้าติดปัญหา

หน่วยความจำเต็ม → Runtime → Restart session

โหลดช้า → รอ ไม่ต้องกดซ้ำ

เซลล์ error → อ่านบรรทัดสุดท้ายของ error ก่อนถาม

Lab 2

ตามรอย prompt: 3 ระดับ วัดด้วยตัวชี้วัดเดียวกัน

โจทย์: สรุปรีวิวร้านกาแฟเป็น JSON · prompt A คลุมเครือ / B สั้น / C เต็ม 4 ส่วน · ไฟล์: Lab2_Trace_Prompt_TH

2

ขั้น ①

2.1 · prompt 3 ระดับ สำหรับโจทย์เดียวกัน

ระดับ	prompt	มีอะไรบ้าง	เทียบกับตอนเช้า
A · คลุมเครือ	"สรุปรีวิวนี้หน่อย: กาแฟดีแต่รอนานมาก"	ไม่ได้บอกด้วยซ้ำว่าอยากได้ JSON	= prompt "ไม่ดี" ในสไลด์ตอนเช้า
B · สั้น	"สรุปรีวิวนี้เป็น JSON: กาแฟดีแต่รอนานมาก"	ขอ JSON แต่ไม่บอกรูปแบบ ไม่มีตัวอย่าง	ระดับกลางที่เพิ่มเข้ามาเพื่อให้เทียบแฟร์
C · เต็ม	บทบาท + คำสั่งรูปแบบ + ตัวอย่าง + คำสั่ง + รีวิว	ครบทุกส่วน (3 ส่วนแรกจะถอดทีละส่วนใน 2.6)	= prompt "ดี" ในสไลด์ตอนเช้า

● ทุกระดับวัดด้วยตัวชี้วัดชุดเดียวกัน: จำนวน token · entropy ของ token แรก · P(ขึ้นต้น JSON) · attention sink · อัตรา parse ผ่าน → รวมเป็นตารางเดียวตอนท้าย

ขั้น ①–②

2.1 · เก็บ prompt เป็น "ส่วน ๆ" ชื่อเดียวกันทุกระดับ

```
REVIEW   = "กาแฟดีแต่รอนานมาก"
SEGMENTS = ["บทบาท", "คำสั่งรูปแบบ", "ตัวอย่าง", "คำสั่ง", "รีวิว"]

def full_parts(role=True, fmt=True, example=True):
    parts = []
    if role:    parts.append(("บทบาท", ROLE))
    if fmt:     parts.append(("คำสั่งรูปแบบ", FORMAT))
    if example: parts.append(("ตัวอย่าง", EXAMPLE))
    parts.append(("คำสั่ง", "ตอนนี้วิเคราะห์รีวิวนี้:"))
    parts.append(("รีวิว", REVIEW))
    return parts

PROMPTS = {
  "A คลุมเครือ": [("คำสั่ง", "สรุปรีวิวนี้หน่อย:"), ("รีวิว", REVIEW)],
  "B สั้น":      [("คำสั่ง", "สรุปรีวิวนี้เป็น JSON:"), ("รีวิว", REVIEW)],
  "C เต็ม":      full_parts(),
}
```

- prompt = รายการ (ชื่อส่วน, ข้อความ) — ส่วนชื่อเดียวกันเทียบกันข้ามระดับได้ เช่น "รีวิว" เป็นข้อความเดียวกันทุกระดับ (ใช้ในกราฟ attention 2.4)
- full_parts(role=False, ...) ปิดทีละส่วน → ใช้ถอด prompt ใน 2.6
- render() ใส่ token พิเศษและ system block (มีวันที่) ให้ — ขั้น ① ของเส้นทาง prompt
- results = {} เก็บตัวชี้วัดทุกอย่างของทุกระดับ ไว้รวมตารางเดียว

ขั้น ④–⑤

2.3 · การกระจายของ token แรก: entropy และ P(ขึ้นต้น JSON)

```
@torch.no_grad()
def next_token_probs(parts):
    enc = encode(render(to_messages(parts)))
    out = model(input_ids=enc.input_ids.to(model.device))
    return torch.softmax(out.logits[0, -1].float(), dim=-
1).cpu()

def entropy_bits(p):              # กระจาย = สูง, เกาะกลุ่ม = ต่ำ
    p = p[p > 0]; return float(-(p * torch.log2(p)).sum())

json_start_ids = [i for i in range(len(tok))
    if tok.decode([i]).lstrip().startswith(("{", "```"))]

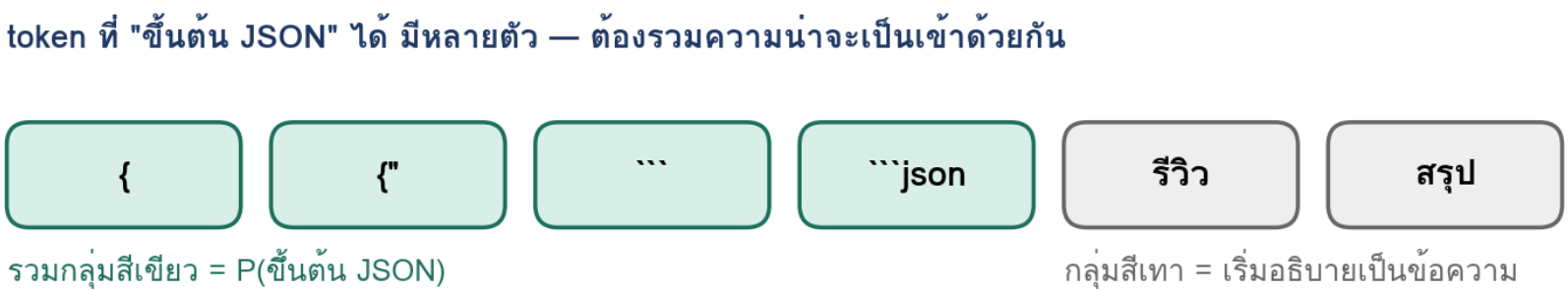
for name, parts in PROMPTS.items():
    p = next_token_probs(parts)
    results[name]["entropy (bit)"]   = entropy_bits(p)
    results[name]["P(ขึ้นต้น JSON)"] =
p[json_start_ids].sum().item()

# ตัวอย่างผล:  A 3.1 bit / 0.01   B 1.4 / 0.72   C 0.4 / 0.96
```

- ขั้น ④–⑤: forward ครั้งเดียว → logits → softmax ได้การกระจายของ token แรก
- entropy คือตัวเลขเดียวที่วัด "กระจาย vs เกาะกลุ่ม" — 0 bit = มั่นใจ token เดียว , 3 bit ≈ ลังเลระหว่าง 8 token
- P(ขึ้นต้น JSON) รวมทุก token ที่ขึ้นต้น { หรือ ``` (มีหลายตัว ไม่ใช่แค่ "{")
- กราฟ top-10 สามระดับวางแกน y เดียวกัน แท่งเขียว = token ที่ขึ้นต้น JSON

ขั้น ⑤

ทำไมต้องรวมกลุ่ม token ที่ขึ้นต้น JSON



- tokenizer อาจมี {" เป็น token เดียว และโมเดลเล็กมักขึ้นต้นด้วย ```json — ถ้าดูแค่ "{" ตัวเดียว จะได้ค่าต่ำทั้งที่โมเดลกำลังจะตอบ JSON

- ตอน parse ให้ตัด ``` ออกก่อน json.loads

ชั้น ④

2.4 · attention ของตำแหน่งสุดท้าย รวมเป็นรายส่วน (ชื่อส่วนชุดเดียวกันทุกระดับ)

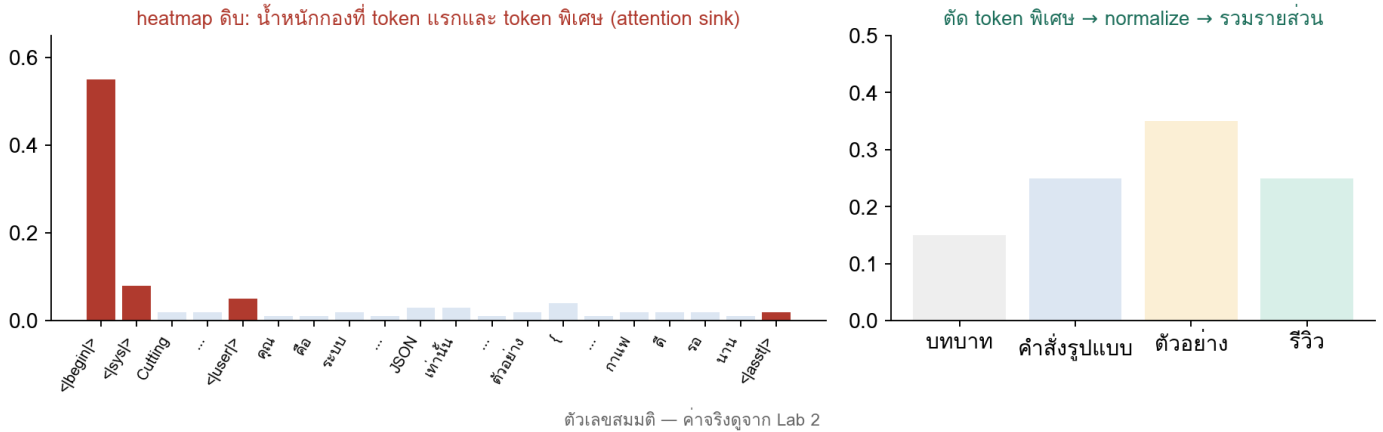
```
@torch.no_grad()
def attention_by_segment(parts, layer=8):
    text = render(to_messages(parts)); enc = encode(text)
    out = model(input_ids=enc.input_ids.to(model.device),
                output_attentions=True)          # ต้อง eager
    att = out.attentions[layer][0].float().mean(0)[-1].cpu()
    #     ↑ ชั้น index 8, เฉลี่ยทุกหัว, แถวของ query ตัวสุดท้าย
    seg_w = {k: 0.0 for k in SEGMENTS + ["template"]}
    sink = 0.0
    for j, tid in enumerate(enc.input_ids[0].tolist()):
        if tid in special_ids: sink += att[j]; continue
        seg = ส่วนที่ token j อยู่ (จาก offset_mapping)
        seg_w[seg] += att[j]
    return normalize(seg_w), sink

# กราฟแท่งกลุ่ม: แกน x = SEGMENTS + template
# แท่ง 3 สี = A / B / C   (ส่วนที่ระดับนั้นไม่มี = 0)
```

- ขั้น ④: attention ดึงข้อมูลจาก token ก่อนหน้า — เราดูว่าดึงจาก "ส่วน" ไหน
- attention ดิบจะกองที่ <|begin_of_text|> (attention sink) → ตัด token พิเศษออกก่อน normalize · notebook วาดกราฟดิบให้ดูก่อน
- ทุกระดับใช้ชื่อส่วนชุดเดียวกัน (SEGMENTS) จึงวางเป็นกราฟแท่งกลุ่มบนแกนเดียวกันได้
- ลองเปลี่ยน layer= เป็น 2, 8, 14 — ชั้นต่างกันมอง prompt ต่างกัน

ขั้น ④

สิ่งที่จะเห็นใน 2.4 (ตัวเลขสมมติ)



ตัวเลขสมมติ — ค่าจริงดูจาก Lab 2

- ซ้าย: attention ดิบต่อ token — น้ำหนักกองที่ token แรก (attention sink) notebook วาดจาก 40 token แรกของระดับ C
- ขวา: หลังตัด token พิเศษและรวมรายส่วน (notebook วาง A/B/C เป็นแท่งกลุ่มบนแกนเดียวกัน)
- ย้ำ: attention เป็น "ร่องรอย" ไม่ใช่คำอธิบาย — หลักฐานจริงคือ 2.5–2.6

ขั้น ⑥

2.5 · generate จริง แล้ววัดอัตรา parse JSON ผ่าน

```
N_SAMPLES = 5          # เพิ่มเป็น 10 ถ้ามีเวลา

def try_parse(s):
    s = strip_fence(s)       # ตัด ```json ... ``` ออก
    try: return json.loads(s)
    except Exception: return None

@torch.no_grad()
def generate(parts, n=N_SAMPLES, temperature=0.7):
    ids = encode(render(to_messages(parts))).input_ids
    torch.manual_seed(0)        # รันซ้ำได้ผลเดิม
    outs = []
    for _ in range(n):
        g = model.generate(ids.to(model.device),
max_new_tokens=80,
                do_sample=True, temperature=temperature,
top_p=0.9,
                eos_token_id=EOT)        # หยุดที่ <|eot_id|>
        outs.append(decode_new(g, ids))
    return outs
# results[name]["parse ผ่าน/5"] = จำนวนที่ parse ผ่าน
```

- ขั้น ⑥: sampling ที่ temperature 0.7 เท่ากันทุกระดับ
- parse ผ่าน = ทั้งคำตอบเป็น JSON (ห่อ ``` ได้) มีคำอธิบายปน = ไม่ผ่าน · ระดับ A ไม่ได้ขอ JSON จึงคาดว่าไม่ผ่านอยู่แล้ว — ใส่ไว้ให้เห็นผลของ "ไม่ระบุรูปแบบ"
- กำหนด seed ไว้ → รันซ้ำได้คำตอบเดิม ทุกระดับได้ "ลูกเต๋า" ชุดเดียวกัน · 5 ครั้งยังผันผวน ถ้ามีเวลาเพิ่มเป็น 10
- ให้อ่านคำตอบจริงของแต่ละระดับด้วย ไม่ใช่ดูแค่ตัวเลข

สรุป

ตารางสรุป: สามระดับ × ตัวชี้วัดทั้งหมด (ตัวอย่างผล — ของจริงดูใน notebook)

ระดับ	token ทั้งหมด	entropy (bit)	P(ขึ้นต้น JSON)	parse ผ่าน/5
A คลุมเครือ	≈ 45	สูง (≈ 3)	≈ 0	0
B สั้น	≈ 48	กลาง (≈ 1–2)	≈ 0.5–0.8	2–4
C เต็ม	≈ 150	ต่ำ (< 1)	≈ 0.9+	4–5

อ่านจาก A → C: token เพิ่ม (ต้นทุน) · entropy ลด (เกาะกลุ่ม) · P(JSON) และ parse เพิ่ม · ถ้า B ใกล้ C แปลว่าโจทย์นี้แค่ 'ขอ JSON' ก็พอ — คำถามที่ Lab 3 จะวัดอย่างเป็นระบบ

ขั้น ⑥

2.6 · ถอดทีละส่วน: ส่วนไหนของ prompt ทำงานจริง

```
configs = [("- บทบาท",         dict(role=False)),
           ("- คำสั่งรูปแบบ", dict(fmt=False)),
           ("- ตัวอย่าง",      dict(example=False))]
abl = {"C ครบทุกส่วน": results["C เต็ม"]}    # แถวอ้างอิง
for name, kw in configs:
    parts = full_parts(**kw)           # เอาออกทีละส่วน
    p     = next_token_probs(parts)
    outs = generate(parts)
    abl[name] = {
      "token ทั้งหมด":   n_tokens(parts),    # นับแบบ 2.2
      "entropy (bit)":   entropy_bits(p),
      "P(ขึ้นต้น JSON)": p[json_start_ids].sum().item(),
      "parse ผ่าน/5":    sum(try_parse(o) is not None
                             for o in outs)}
pd.DataFrame.from_dict(abl, orient="index")

# ตัวอย่างผล        token  entropy  P(JSON)  parse
# C ครบทุกส่วน        150      0.4     0.96      5
# - บทบาท             140      0.5     0.94      5
# - คำสั่งรูปแบบ      125      1.6     0.55      2
```

- เปลี่ยนตัวแปรทีละอย่างจากระดับ C — ส่วนที่ถอดแล้ว P(JSON)/parse ตกมากที่สุด = ส่วนที่ "ทำงาน" จริง
- โจทย์นี้ต้องการ "รูปแบบ" ไม่ใช่ "น้ำเสียง" — คาดว่าคำสั่งรูปแบบและตัวอย่างสำคัญ ส่วนบทบาทช่วยน้อย
- เทียบกับ attention ใน 2.4: ส่วนที่ถูกมองมาก แต่ถอดแล้วผลไม่ตก แปลว่า attention ไม่ใช่คำอธิบาย
- **ชิ้นงาน: notebook + ข้อสรุป 3 บรรทัด (ส่วนไหนถอดแล้วผลตกมากสุด · สอดคล้องกับ attention ไหม · ผลกับรีวิวของตัวเอง)**

สรุป

สรุป Lab 2

prompt ทุกระดับคือ token ที่ต่อกันข้างหน้า
ต่างกันที่ "ให้อะไรกับ attention ดึง" และต้นทุน token ที่จ่าย

entropy ของ token แรกคือตัวเลขเดียวที่บอกว่า prompt ทำให้โมเดล "เกาะกลุ่ม" แค่ไหน attention บอกว่าโมเดล "มอง" อะไร · การถอดทีละส่วนบอกว่าอะไร "มีผลจริง"

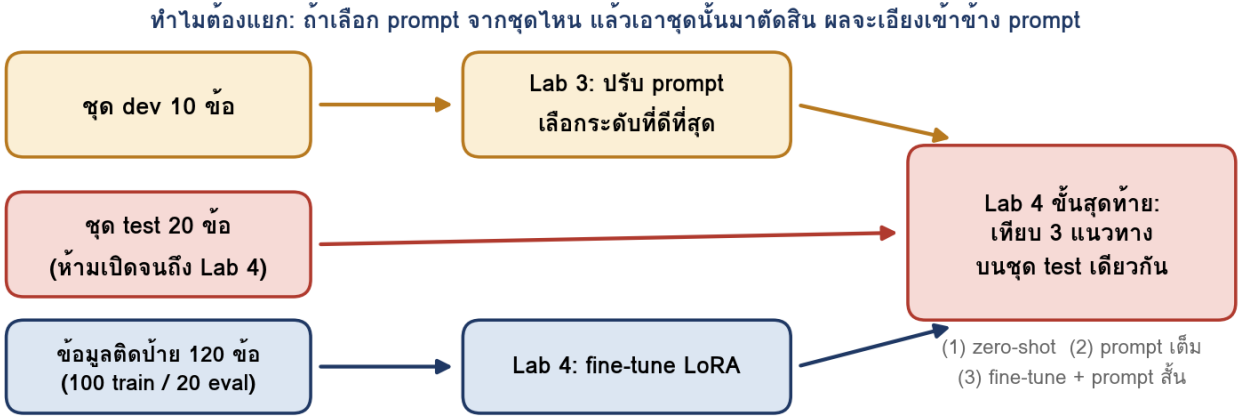
Lab 3

Prompt engineering อย่างเป็นระบบ

โจทย์ (ข) สกัดสินค้า+ราคา → JSON · โจทย์ (ค) จัดหมวดคำร้องเรียน 4 หมวด · ไฟล์: Lab3_Prompt_Engineering_TH

3

กติกาสำคัญ: แยกชุด dev / test



- dev 10 ข้อ ใช้ปรับ prompt ใน Lab 3
- test 20 ข้อ ห้ามเปิดจนถึงขั้นสุดท้ายของ Lab 4
- ถ้าเลือก prompt จากชุดไหนแล้วเอาชุดนั้นมาตัดสิน ผลจะเอียงเข้าข้าง prompt

ข้อมูล

ข้อมูลใน notebook (ฝังไว้แล้ว ไม่ต้องอัปโหลด)

```
DATA = json.loads(r"""{ ... }""")    # ฝังในไฟล์

DATA["extract"]["dev"][0]
# {"text": "หูฟังไร้สาย SoundPro X2 ราคา 1,290 บาท ส่งฟรี",
#  "product": "หูฟังไร้สาย SoundPro X2",
#  "price": 1290}

DATA["classify"]["dev"][0]
# {"text": "สั่งของมา 10 วันแล้วยังไม่ได้รับเลย ...",
#  "label": "การจัดส่ง"}

DATA["classify"]["categories"]
# ["การจัดส่ง", "สินค้าชำรุด",
#  "การชำระเงิน", "บริการลูกค้า"]

# ขนาดต่อโจทย์: dev 10 · test 20 · train 120
```

- โจทย์ (ข) extract: input = ประกาศขาย → output = {"product": ..., "price": ตัวเลข}
- โจทย์ (ค) classify: input = คำร้องเรียน → output = ชื่อหมวด 1 หมวด
- dev/test เขียนด้วยมือ · train 120 ข้อ สร้างจากแม่แบบ (สังเคราะห์) ใช้ใน Lab 4
- โจทย์เสริม (ก) สรุปข่าว 1 ประโยค สำหรับกลุ่มที่เสร็จเร็ว — ชุด dev 10 ข้อ และเกณฑ์ 0/1/2 ผู้สอนแจกแยก (ไม่อยู่ใน DATA)

ขั้นที่ 1–2

3.1 · ฟังก์ชันเรียกโมเดล และวัดผล (งานต่างชนิดวัดคนละแบบ)

```
@torch.no_grad()
def ask(system, user, max_new_tokens=80):
    msgs = ([{"role": "system", "content": system}]
            if system else [])
    msgs += [{"role": "user", "content": user}]
    text = tok.apply_chat_template(msgs, tokenize=False,
                                   add_generation_prompt=True)
    ids = tok(text, add_special_tokens=False,
              return_tensors="pt").input_ids.to(model.device)
    g = model.generate(ids, max_new_tokens=max_new_tokens,
                       do_sample=False, eos_token_id=EOT)  # greedy
    out = tok.decode(g[0, ids.shape[1]:], skip_special_tokens=True)
    return out.strip(), ids.shape[1]     # คำตอบ, ความยาว prompt

def score_extract(output, item):       # 0 … 1
    obj = parse_json_obj(output)       # JSON → dict
    if obj is None: return 0.0
    return product_score(obj, item) + price_score(obj, item)

def score_classify(output, item):      # 0 หรือ 1
    ok = first_answer(output) == item["label"]  # บรรทัดแรกพอดี
    return 1.0 if ok else 0.0
```

- ask() คืน (คำตอบ, จำนวน token ของ prompt) — เก็บความยาว prompt ไว้เทียบต้นทุน
- (ข) product ตรงเป๊ะ 0.5 / ตรงบางส่วน 0.25 + price 0.5
- (ค) บรรทัดแรกต้องเป็นชื่อหมวดที่ถูกพอดี — ตอบยาวเป็นย่อหน้า = 0 (นับแยกในคอลัมน์ "ถูกหมวดแต่ตอบยาว")
- "ตอบตรงรูปแบบ" (format_ok): (ข) ทั้งคำตอบต้องเป็น JSON ที่มี product, price · (ค) บรรทัดแรกเป็นชื่อหมวดใดหมวดหนึ่ง
- **ทำไมเข้ม: คำตอบถูกส่งต่อให้โปรแกรมอ่านเอง — โปรแกรมอ่าน "น่าจะเป็นหมวดการจัดส่งนะคะ" ไม่ออก**

ขั้นที่ 3

3.2 · ไล่ระดับ prompt ทีละตัวแปร (โจทย์ ค)

```
CAT_STR = " / ".join(CATS)
EX_C  = [("ของมาถึงช้ากว่ากำหนดสามวัน", "การจัดส่ง"),
         ("แก้วที่ได้รับมีรอยร้าว", "สินค้าชำรุด")]
FEW_C = "\n".join(f"คำร้องเรียน: {t}\nหมวด: {a}"
                  for t, a in EX_C)
HEAD  = (f"จัดหมวดหมู่คำร้องเรียนต่อไปนี้ "
         f"เลือกหนึ่งหมวดจาก [{CAT_STR}] ตอบเฉพาะชื่อหมวด")

LEVELS_C = {   # (system หรือ None, ฟังก์ชันสร้าง user)
 "L0 zero-shot":       (None, lambda x:
      f"คำร้องเรียนนี้อยู่ในหมวดไหน: {x}"),
 "L1 + รูปแบบ output": (None, lambda x:
      f"{HEAD}\nคำร้องเรียน: {x}\nหมวด:"),
 "L2 + few-shot":      (None, lambda x:
      f"{HEAD}\n{FEW_C}\nคำร้องเรียน: {x}\nหมวด:"),
 "L3 + บทบาท":         (ROLE_C, lambda x:    # + system
      f"{HEAD}\n{FEW_C}\nคำร้องเรียน: {x}\nหมวด:"),
}   # ใน notebook เขียนข้อความเต็มแทน HEAD / ROLE_C
```

- L0 → L1 เพิ่ม "รูปแบบ output" (รายชื่อหมวด + ตอบเฉพาะชื่อหมวด)
- L1 → L2 เพิ่ม few-shot 2 ตัวอย่าง
- L2 → L3 เพิ่ม บทบาท ใน system prompt (ข้อความ user เหมือน L2 ทุกตัวอักษร — เปลี่ยนอย่างเดียวจริง ๆ)
- เปลี่ยนทีละอย่าง → รู้ว่าคะแนนที่เพิ่มมาจากอะไร (ตรงกับตารางหลักการ 5 ข้อตอนเช้า)
- โจทย์ (ข) มี LEVELS_B โครงเดียวกัน แต่ระบุรูปแบบ JSON

ขั้นที่ 3

3.2 · รันทุกระดับบนชุด dev แล้วสรุปเป็นตาราง

```
results = []
for task, levels in [("extract", LEVELS_B),
                     ("classify", LEVELS_C)]:
    for name, (system, fn) in levels.items():
        r = evaluate(task, system, fn, "dev")    # → dict
        results.append({"โจทย์": task,
                        "ระดับ prompt": name, **r})
df = pd.DataFrame(results); df

# คอลัมน์: คะแนน dev · ตอบตรงรูปแบบ ·
#          token ของ prompt · ถูกหมวดแต่ตอบยาว
# ตัวอย่าง (ค่าจริงอาจต่าง)
# classify L0 zero-shot     0.10   1/10   40   6
# classify L1 + รูปแบบ      0.70  10/10   75   0
# classify L2 + few-shot    0.80  10/10  120   0

evaluate("classify", *LEVELS_C["L0 zero-shot"],
         "dev", verbose=True)    # ดูคำตอบจริงทีละข้อ
```

- ตารางเดียวตอบได้ว่า: โจทย์ไหน prompt ระดับใดก็พอ และโจทย์ไหน L3 แล้วยังไม่ถึงเป้า
- "ถูกหมวดแต่ตอบยาว" สูงใน L0 = โมเดลรู้คำตอบ แต่ไม่ทำตามรูปแบบ → สิ่งที่ L1 แก้
- ดูคอลัมน์ token ด้วย — คะแนนเพิ่ม 0.05 แต่ prompt ยาวขึ้น 2 เท่า คุ้มไหม?
- verbose=True พิมพ์คำตอบจริงทีละข้อ: ตอบยาวเกิน? ตอบหลายหมวด? JSON ผิดรูป?
- **เป้า = คะแนน dev ≥ 0.9 · โจทย์ที่ L3 ยังต่ำกว่าเป้า = ตัวเลือกสำหรับ fine-tune ใน Lab 4**

ขั้นที่ 3

3.3 · (แถวแยก) chain-of-thought กับโจทย์ (ค)

```
EX_C_COT = [
  ("ของมาถึงช้ากว่ากำหนดสามวัน",
   "ปัญหาอยู่ที่ความล่าช้าของการส่ง", "การจัดส่ง"),
  ("แก้วที่ได้รับมีรอยร้าว", "ตัวสินค้าเสียหาย", "สินค้าชำรุด")]
FEW_C_COT = "\n".join(
  f"คำร้องเรียน: {t}\nเหตุผล: {r}\nหมวด: {a}"
  for t, r, a in EX_C_COT)

def cot_user(x):
    return (f"จัดหมวดหมู่ ... เลือกจาก [{CAT_STR}]\n"
            f"เขียนบรรทัด 'เหตุผล: <1 ประโยค>' ก่อน "
            f"แล้วปิดท้ายด้วย 'หมวด: <ชื่อหมวด>'\n"
            f"{FEW_C_COT}\nคำร้องเรียน: {x}")

def score_classify_cot(output, item):   # เข้มเท่า L2
    m = re.search(r"หมวด\s*:\s*(.+)", output)   # บรรทัด หมวด: แรก
    return 1.0 if m and first_answer(m.group(1)) \
                   == item["label"] else 0.0
# พิมพ์ทั้งคะแนนและความยาวคำตอบ (token) ของ L2 เทียบ CoT
```

- ทำเฉพาะ (ค) เพราะ (ข) ต้องตอบ JSON เท่านั้น ใส่เหตุผลไม่ได้ · ให้คะแนนเข้มเท่ากับ L2 (ชื่อหมวดหลัง "หมวด:" ต้องตรงพอดี)
- ตัวอย่าง few-shot ต้องมีบรรทัดเหตุผลด้วย (FEW_C_COT) ไม่งั้นโมเดลจะเลียนแบบแล้วข้ามการอธิบาย · ตัววัดผลดูเฉพาะหลัง "หมวด:"
- notebook พิมพ์คะแนนและความยาวคำตอบของ L2 กับ CoT คู่กัน — โมเดล 1B มักได้ประโยชน์จาก CoT น้อย แต่จ่าย token ตอบมากขึ้นหลายเท่า
- กลไก (ตอนเช้า): ให้โมเดลเขียนขั้นกลางออกมา = token ถัดไปมีข้อมูลระหว่างทางให้ดึง

ขั้นที่ 4

3.4 · เทียบกับ ChatGPT และ Claude (ผ่านหน้าเว็บ ไม่ต้องใช้ API)

- **เลือก 1 โจทย์ × 3 ข้อจากชุด dev · ยิงเฉพาะ L0 กับ L3 · เข้า 2 โมเดล = 12 ครั้ง**
  - เซลล์ 3.4 ไม่ใช่ผลลัพธ์ — มันพิมพ์ prompt 6 ชุด (3 ข้อ × 2 ระดับ) ให้คัดลอกข้อความระหว่างเส้นประทั้งก้อนไปวางในหน้าเว็บ
  - ถ้ามี [system] ให้วางไว้บรรทัดแรกของข้อความ แล้วต่อด้วย user prompt ในข้อความเดียวกัน (หน้าเว็บไม่มีช่อง system)
- **เปิดแชทใหม่ทุกครั้ง และปิด memory / custom instructions**
  - ไม่งั้นบริบทจากแชทก่อนหน้าจะปนเข้ามาในผล
- **ใต้แต่ละ prompt มีคำตอบของ Typhoon2 + คะแนนไว้แล้ว · เติมคอลัมน์ ChatGPT / Claude ในตารางท้ายเซลล์ (1 = ถูก, 0 = ผิด)**
  - คาดว่าจะเห็น: โมเดลใหญ่ต้องการ prompt น้อยกว่า แต่หลักการ "ระบุรูปแบบ" ยังมีผลกับทุกโมเดล

● ถ้าเวลาไม่พอ ผู้สอนสาธิตบนจอแทน 1 โจทย์ · ต้องมีบัญชี ChatGPT/Claude แบบฟรี

ขั้นที่ 5

3.5 · บันทึก prompt ที่ดีที่สุดไปใช้ใน Lab 4

```
best = {}
for task, levels in [("extract", LEVELS_B),
                     ("classify", LEVELS_C)]:
    cand = df[(df["โจทย์"] == task) &
              (df["ระดับ prompt"] != "L0 zero-shot")]
    sub  = cand.sort_values("คะแนน dev", ascending=False,
                            kind="stable").iloc[0]
    name = sub["ระดับ prompt"]; system, fn = levels[name]
    best[task] = {"level": name, "system": system,
                  "user_template": fn("{TEXT}"),
                  "dev_score": float(sub["คะแนน dev"])}

json.dump(best, open("best_prompts.json", "w",
          encoding="utf-8"), ensure_ascii=False)
# Lab 4 อ่านไฟล์นี้เป็นแนวทาง (2) "prompt เต็ม"
```

- เลือกระดับที่คะแนน dev สูงสุดจาก L1–L3 (L0 เป็นเส้นฐานใน Lab 4 อยู่แล้ว) · เสมอกัน → เลือกระดับที่สั้นกว่า
- เก็บ user prompt เป็นแม่แบบที่มี {TEXT} ให้แทนที่ตอนใช้
- ไฟล์อยู่ใน runtime ของ Colab — หายเมื่อปิด ต้องดาวน์โหลดจากแผง Files ไว้แล้วอัปโหลดใน Lab 4
- **ชิ้นงาน Lab 3: ตารางผลบนชุด dev + prompt ที่ดีที่สุดเก็บเข้า prompt library ของตัวเอง**

สรุป

สรุป Lab 3

วัดผลด้วยชุด dev และเปลี่ยน prompt ทีละตัวแปร
→ รู้ว่าอะไรช่วยจริง และคุ้มกับ token ที่เพิ่มไหม

โจทย์ที่ prompt เต็ม (L3) แล้วยังไม่ถึงเป้า คือตัวเลือกสำหรับ fine-tune ใน Lab 4
ห้ามเปิดชุด test จนกว่าจะถึงขั้นสุดท้ายของ Lab 4

Lab 4

เตรียมข้อมูลและ fine-tune ด้วย LoRA

แปลงข้อมูลเป็นบทสนทนา + สร้าง label (คิด loss เฉพาะคำตอบ) · LoRA rank 16 ใน q, k, v, o · เทรน 1–3 นาที · เทียบ 3 แนวทางบนชุด test พร้อมค่า p · ไฟล์: Lab4_LoRA_Finetune_TH

4

ขั้นที่ 0

4.0 · ติดตั้งและโหลดโมเดล (fp16 ธรรมดา ไม่ต้อง 4-bit)

```
!pip install -q -U "transformers>=4.51,<5" \
     "peft>=0.12" accelerate datasets

MODEL = "scb10x/llama3.2-typhoon2-1b-instruct"
tok = AutoTokenizer.from_pretrained(MODEL)
EOT = tok.convert_tokens_to_ids("<|eot_id|>")    # token จบ
if tok.pad_token is None:
    tok.pad_token = "<|eot_id|>"     # ใช้เติมความยาว batch

model = AutoModelForCausalLM.from_pretrained(
    MODEL, torch_dtype=torch.float16).to("cuda")
```

- peft = ไลบรารี LoRA ของ Hugging Face · datasets = ส่งข้อมูลเข้า Trainer
- โมเดล 1B แบบ fp16 ใช้หน่วยความจำ ≈ 2.5 GB — T4 มี 15 GB เหลือเฟือ จึงไม่ต้อง 4-bit
- QLoRA (4-bit) หรือ Unsloth จำเป็นเมื่อโมเดล 7B ขึ้นไป — หลักการ LoRA เหมือนเดิมทุกอย่าง
- EOT = token จบคำตอบ ใช้ทั้งตอนเทรน (ต่อท้ายคำตอบ) และตอน generate (หยุดตรงนี้) · pad token ใช้เติมข้อมูลที่ยาวไม่เท่ากันใน batch

ขั้นที่ 1

4.1 · แปลงข้อมูลเป็นบทสนทนา แล้วสร้าง label

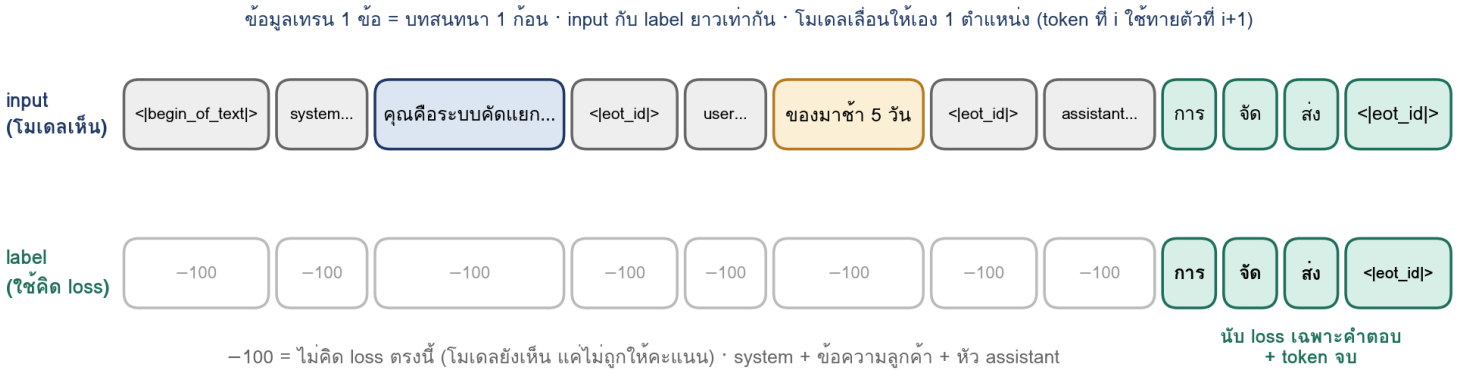
```
def build_example(it):
    # system สั้น + ข้อความ + หัว assistant
    ids_prompt = encode_prompt(SYSTEM, it["text"])[0].tolist()
    # คำตอบ + token จบ <|eot_id|>
    ans = tok(answer_of(it), add_special_tokens=False)
    ids_answer = ans.input_ids + [EOT]
    return {
      "input_ids": ids_prompt + ids_answer,
      "attention_mask": [1] * (len(ids_prompt)
                               + len(ids_answer)),
      "labels": [-100] * len(ids_prompt)    # ไม่คิด loss
                + ids_answer}                # คิด loss

items = list(DATA[TASK]["train"])            # 120 ข้อ
random.Random(0).shuffle(items)
train_ex = [build_example(x) for x in items[:100]]
eval_ex  = [build_example(x) for x in items[100:]]
```

- input = ทั้งก้อน (โมเดลเห็นทุก token) · label = ก้อนเดียวกัน แต่ส่วนที่ไม่ใช่คำตอบเป็น −100
- −100 = "ไม่นับ loss ตรงนี้" → โมเดลเรียนวิธีตอบ ไม่ได้เรียนเดาว่าลูกค้าจะบ่นอะไร
- นับ <|eot_id|> ด้วย → โมเดลเรียนว่าตอบเสร็จแล้วต้องหยุด
- SYSTEM สั้นมาก ("คัดแยกคำร้องเรียนเป็นหมวด") ไม่ต้องบอกรายชื่อหมวด — โมเดลเรียนจากตัวอย่างเอง · ต้องใช้ตัวเดียวกันตอนใช้งาน
- eval 20 ข้อ = ดู overfitting ระหว่างเทรน (คนละชุดกับ test)

ขั้นที่ 1

input กับ label ของข้อมูลเทรน 1 ข้อ



- เทียบโมเดลภาพ: input = ภาพ, label = ชื่อคลาส — ที่นี่ input = บทสนทนาทั้งก้อน, label = token ของคำตอบ
- ส่วนที่เป็น −100 ยังอยู่ใน input ให้ attention ดึงได้ แค่ไม่ถูกนับ loss
- การตัดคำไทยเป็น token ในภาพเป็นตัวอย่าง ของจริงดูผล ③ ใน notebook

ขั้นที่ 2

4.2 · ใส่ LoRA แล้วนับพารามิเตอร์ที่เทรนจริง

```
from peft import LoraConfig, get_peft_model
model = get_peft_model(model, LoraConfig(
    r=16, lora_alpha=16, lora_dropout=0.0,
    target_modules=["q_proj", "k_proj",
                    "v_proj", "o_proj"],
    task_type="CAUSAL_LM"))        # set_seed(0) ก่อนหน้านี้
for p in model.parameters():           # ตาราง LoRA
    if p.requires_grad: p.data = p.data.float()

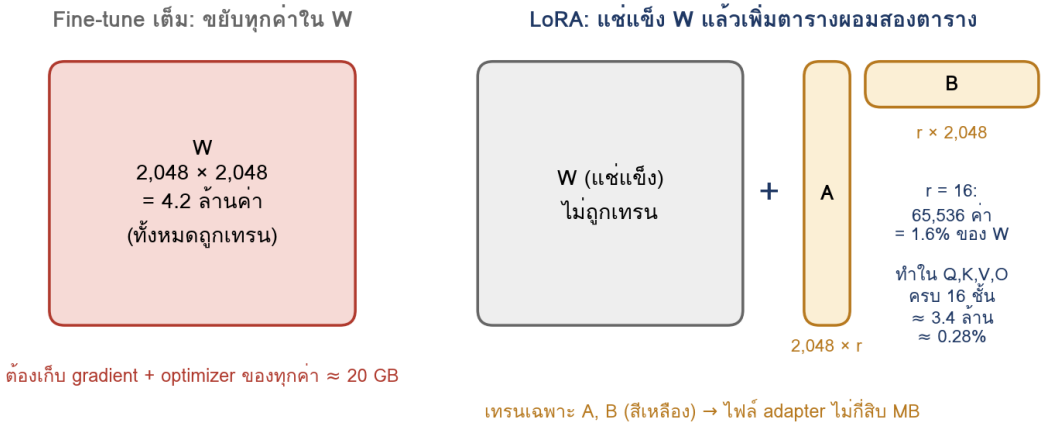
trainable = sum(p.numel() for n, p in
    model.named_parameters() if "lora_" in n)
total = sum(p.numel() for p in model.parameters())
# ≈ 3.41 ล้าน จาก ≈ 1,239 ล้าน = 0.28%

# คำนวณมือ: (Q + O) + (K + V) ต่อชั้น × 16
H, kv, r = 2048, 512, 16
print((2*(H*r + r*H) + 2*(H*r + r*kv)) * 16)
```

- ให้นักศึกษาคำนวณก่อนรัน แล้วเทียบกับที่โค้ดพิมพ์ — ควรได้ ≈ 3.4 ล้าน (≈ 0.28%)
- K, V เล็กกว่า Q, O เพราะกว้างแค่ 512 (GQA: 8 หัว × 64)
- ตาราง LoRA เก็บเป็น fp32 ส่วนโมเดลหลักยัง fp16 → เทรนแบบ fp16 บน T4 ได้เสถียร
- lora_alpha = ตัวคูณขนาดของการเปลี่ยนแปลง ใช้ค่าเท่า r เป็นค่าเริ่มต้นที่ปลอดภัย

ขั้นที่ 2

ทบทวน: LoRA เทรนแค่ A และ B



- W ถูกแช่แข็ง
- เทรนเฉพาะตารางผอม A, B ใน Q, K, V, O ทั้ง 16 ชั้น
- ผลลัพธ์คือไฟล์ adapter ราว 14 MB แยกจากโมเดลหลัก

ขั้นที่ 3

4.3 · เทรนด้วย Trainer ของ Hugging Face

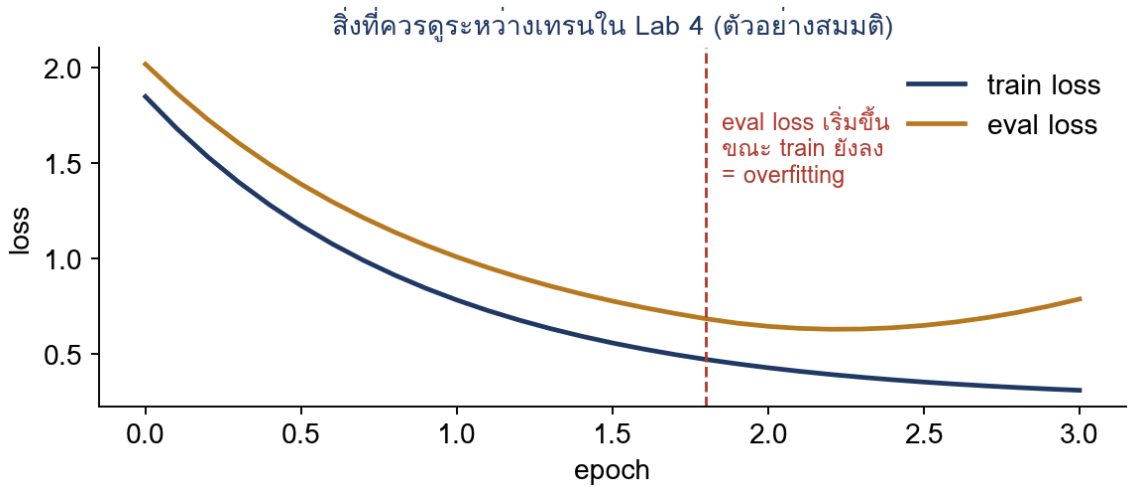
```
args = TrainingArguments(output_dir="lab4_out",
    num_train_epochs=3, per_device_train_batch_size=8,
    learning_rate=2e-4, warmup_steps=5,
    logging_steps=5, eval_strategy="steps",
    eval_steps=5,                  # eval loss ทุก 5 step
    save_strategy="no", fp16=True, # T4 ไม่มี bf16
    label_names=["labels"], report_to="none", seed=0)

trainer = Trainer(model=model, args=args,
    train_dataset=Dataset.from_list(train_ex),
    eval_dataset=Dataset.from_list(eval_ex),
    data_collator=DataCollatorForSeq2Seq(
        tok, padding=True, label_pad_token_id=-100))
trainer.train()      # 39 step ≈ 1-3 นาทีบน T4
trainer.evaluate()   # วัด eval ครั้งสุดท้าย
# จากนั้นวาดกราฟ train/eval loss จาก log
```

- ไม่ต้องใช้ฟังก์ชันพิเศษเรื่อง mask — label ตั้ง −100 ไว้แล้วใน 4.1
- DataCollatorForSeq2Seq เติม padding ให้ยาวเท่ากันใน batch และเติม label ด้วย −100 (ไม่คิด loss ที่ padding)
- 3 epoch × 100 ข้อ / batch 8 ≈ 39 step · eval ทุก 5 step → กราฟ eval มีจุดพอเห็นแนวโน้ม
- learning_rate 2e-4 สูงกว่า fine-tune เต็ม (1e-5) เพราะ LoRA เทรนตารางเล็กที่เริ่มจากศูนย์
- ระหว่างเทรนพิมพ์ train / eval loss ทุก 5 step บรรทัดละครั้ง แล้ววาดกราฟตอนจบ

ขั้นที่ 3

ระหว่างเทรน ดูอะไร



- train loss ควรลง
- eval loss ควรลงตาม — ถ้าเริ่มนิ่งหรือขึ้นขณะ train ยังลง = เริ่มท่องจำ (overfitting) → ลด epoch หรือเพิ่มข้อมูล
- ในแล็บนี้ eval มาจากแม่แบบเดียวกับ train จึงมักลงด้วยกันจนใกล้ 0 — ไม่ได้แปลว่าเก่งจริง ตัวตัดสินคือ test ใน 4.4

ขั้นที่ 4

4.4 · เปิดชุด test ครั้งแรก เทียบ 3 แนวทาง

```
def ask(system, user, adapter=True):
    ctx = nullcontext() if adapter \
          else model.disable_adapter()   # ปิด LoRA
    with ctx: return generate(system, user)

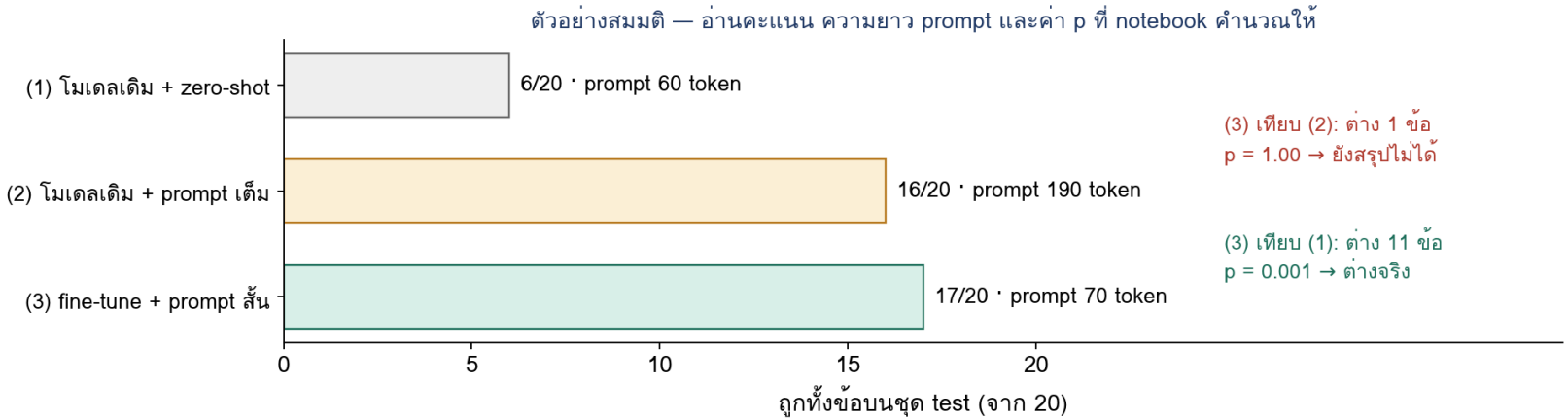
ARMS = {"(1) เดิม + zero-shot":  (*L0,  False),
        "(2) เดิม + prompt เต็ม": (*L3,  False),
        "(3) fine-tune + สั้น":   (SYSTEM, lambda x: x, True)}
for name, (system, fn, adapter) in ARMS.items():
    for it in DATA[TASK]["test"]:            # 20 ข้อ
        out, n = ask(system, fn(it["text"]), adapter)
        # เก็บ: คะแนน · ถูกทั้งข้อ · ตอบตรงรูปแบบ · token

compare(arm2, arm3); compare(arm1, arm3)
# (3) เทียบ (2): ต่าง +1 ข้อ ... p = 1.000
#                → ยังสรุปไม่ได้
```

- ชุด test 20 ข้อ ไม่เคยใช้ปรับ prompt จึงตัดสินได้ยุติธรรม
- ทั้งสามแนวทางใช้โมเดลหลักตัวเดียวกัน ในหน่วยความจำ — (1)(2) แค่ปิด LoRA ชั่วคราว
- (2) = prompt ที่ดีที่สุดจาก best_prompts.json (ไม่มีไฟล์ → ใช้ L3 ที่ฝังไว้)
- (3) ต้องใช้ SYSTEM ตัวเดียวกับตอนเทรน ไม่งั้นไม่ยุติธรรมกับ fine-tune
- **สรุปตามค่า p: < 0.05 = ต่างจริง · 0.05–0.20 = มีแนวโน้ม · มากกว่านั้น = ยังสรุปไม่ได้**

ขั้นที่ 4

อ่านผลอย่างไร (ตัวอย่างสมมติ)



- fine-tune ชนะ prompt เต็มแค่ 1 ข้อ p = 1.00 → ยังสรุปไม่ได้ว่าคะแนนดีกว่า
- แต่ prompt สั้นกว่าเกือบ 3 เท่า → ถ้าเรียกวันละหมื่นครั้ง ต้นทุนต่างกันมาก
- คำตอบไม่จำเป็นต้องเป็น "fine-tune ดีกว่า" เสมอไป — prompt ชนะก็เป็นผลที่ถูกต้องได้

ขั้นที่ 5

4.5 · ดูข้อที่ตอบผิด และบันทึก adapter

```
# ดูข้อที่แนวทาง (3) ตอบไม่ถูกทั้งข้อ
wrong = [x for x in rows[arm3] if x["score"] < 1]
for x in wrong[:5]:
    print(f"[{x['score']:.2f}] {x['text'][:50]}")
    print(f"       ตอบ: {x['output'][:60]!r}")

# บันทึกเฉพาะ LoRA adapter (≈ 14 MB)
model.save_pretrained("lora_adapter")
!du -sh lora_adapter
```

- ดูข้อที่ผิด: ผิดเพราะข้อมูลเทรนไม่มีเคสแบบนี้? หรือเพราะ label กำกวม? → บอกว่าควรเพิ่มข้อมูลแบบไหน
- adapter = ไฟล์เล็ก สลับ "ทักษะ" ได้ โดยไม่ต้องเก็บโมเดลใหม่ทั้งตัว
- ถ้าจะแชร์ adapter: license ของ Llama 3.2 กำหนดให้ระบุ "Built with Llama"
- **ชิ้นงาน Lab 4: notebook + ตารางเทียบ 3 แนวทางบนชุด test + ข้อสรุปตามค่า p**

สรุป

สรุป Lab 4

LoRA rank 16 ใน q, k, v, o เทรนเพียง ≈ 0.3% ของโมเดล แต่เปลี่ยนพฤติกรรมได้ — ถ้าข้อมูลอยู่ในรูปที่โมเดลเคยเห็น

ข้อมูล = บทสนทนา (system + ข้อความ + คำตอบ) · label = −100 (ไม่คิด loss) ทุกตำแหน่งยกเว้นคำตอบ · ใช้ system prompt เดิมตอนใช้งาน
การเทียบที่ยุติธรรมต้องใช้ชุด test ที่ไม่เคยใช้ปรับ prompt และดูค่า p ก่อนสรุป

การบ้าน

การบ้าน: ทำวงจรเดียวกันนี้กับงานภาษาไทยที่คุณเลือกเอง

- **โจทย์: พิสูจน์ด้วยตัวเลขว่างานของคุณควรใช้ prompt หรือ fine-tune**
  - รายละเอียดทั้งหมดอยู่ในสไลด์ "การบ้านสัปดาห์ 3" (แยกไฟล์)
- **ข้อมูล 3 ชุด: dev 20 · test 30 · train 150–300 ข้อ**
  - dev/test เขียนเองหรือข้อมูลจริงที่ตรวจแล้ว · train ให้ ChatGPT/Claude ช่วยร่างได้แต่ต้องตรวจทุกข้อ
- **notebook ตั้งต้น HW_Starter_TH.ipynb — โค้ดชุดเดียวกับ Lab 3–4 แก้แค่ 3 เซลล์**
  - ไฟล์ตัวอย่างข้อมูล my_task_example.json ใช้ดูรูปแบบ
- **ส่ง: notebook ที่รันครบ · my_task.json · hw_results.json · memo 1 หน้า · สไลด์ 4 หน้า**
  - งานเดี่ยว · ส่งก่อนเริ่มคาบสัปดาห์หน้า · วันที่ 1 ส่งชื่องาน + ตัวอย่าง 5 ข้อ + เกณฑ์ให้คะแนน

prompt ชนะก็ได้คะแนนเต็ม ถ้าการทดลองยุติธรรมและเหตุผลอิงหลักฐาน

สรุปวัน

สิ่งที่ทำบ่ายนี้ ↔ ทฤษฎีช่วงเช้า

1 Lab 2

เส้นทาง prompt 6 ขั้น · logits/softmax · attention (และ attention sink) · sampling

→ prompt ดีทำให้ token แรก "เกาะกลุ่ม"

2 Lab 3

หลักการ prompt 5 ข้อ · in-context learning · CoT กับโมเดลเล็ก · dev/test

→ วัดผลได้ ไม่ใช่ลองผิดลองถูก

3 Lab 4

chat template · input กับ label (−100) · LoRA 0.3% · overfitting · ค่า p

→ fine-tune เปลี่ยน weight, prompt เปลี่ยน input

สะพานสู่สัปดาห์ 4

สัปดาห์หน้า: RAG

วันนี้เราใส่บริบทลงใน prompt ด้วยมือ และเห็นแล้วว่าทำไมมันได้ผล
(attention ดึงได้เฉพาะสิ่งใน context)
สัปดาห์หน้าจะให้ระบบไปค้นบริบทมาใส่เอง

embedding model (แปลงทั้งข้อความเป็น vector — คนละตัวกับชั้น embedding ที่ผ่าวันนี้) · vector database · chunking · RAG bot จากเอกสารไทย